AnalysisTools User Manual

RNDr. Karel Šindelka, Ph.D.
k.sindelka@gmail.com

Version 4.x (11th August 2026)

Contents

1. Introduction

AnalysisTools is a set of utilities to (mainly) analyse trajectories of particle-based simulations. The utilities (including one from the DLMESO simulation package) can be roughly divided into four types:

- utilities to calculate system-wide properties; e.g., pair correlation functions, densities, or orientational order parameters
- utilities to calculate per-molecule or per-aggregate properties (where aggregate stands for any supramolecular structure); e.g., shape descriptors, persistence lengths, or membrane properties such as bending rigidity and tail interdigitation
- utilities to manipulate or generate configurations; e.g., create initial configurations from scratch, add molecules to an existing system, or join two systems together
- helper utilities to analyse text files; e.g., calculate averages and standard deviations of a data series

The `Examples` directory contains examples showcasing capabilities of individual utilities.

1.1 Installation

Requirements

- `GSL`, the GNU Scientific Library
- `cmake` to generate files necessary to build AnalysisTools
- `C` and `Fortran` compilers

To compile the utilities, run the `compile.sh` script in the `AnalysisTools` root directory. By default, the build directory is `build` in the `AnalysisTools` root; to use a different directory add it as an argument to the script, e.g., `./compile.sh custom_dir`. In both cases, the executables will be located in the `bin/` subdirectory of the build directory. To compile only a single utility, run `make <utility name>` directly from within the build directory.

2. Format of input/output files

This chapter describes several file types used by many of the AnalysisTools utilities. Output files for the utilities themselves are described in their respective sections in Utilities (chapter 3).

Input files are divided into two categories: structure and coordinate files. The structure files contain information about the system composition, that is, number of beads and molecules, bead properties (name, mass, charge, etc.), molecule properties (name, number and types of beads, connectivity, etc.). The coordinate files contain individual timesteps with beads' coordinates (and, possibly, velocities, forces, etc.). While most files can be used as both a structure and a coordinate file, using a separate structure file may be essential. For example, an `xyz` file (see below) may be used as both, but because it only provides bead names, using a separate structure file would provide additional information (bead and/or molecule properties).

For working with aggregates, AnalysisTools' `agg` file format (see below) describes their compositions in terms of molecule numbers taken from the structure file used to generate it. The Aggregates (section 3.3) utility creates the `agg` file.

2.1 Structure files

The following subsections describe the supported formats; browse the `Examples` directory for example files. Most utilities can load a specific structure file using the `-i <file> [type]` option (with optional `type` provided if it doesn't have expected extension); if the option is not used, the coordinate file is typically used as a structure file as well. The type of the input coordinate file can similarly be overridden using the `-ft <type>` option. Recognised type strings are listed in each subsection below.

Importantly, many of the AnalysisTools utilities work (either fundamentally or optionally) with types of beads and/or molecules. In these cases, names are used specify the bead/molecule types. While many input files support naming beads and molecules, not all do (in some formats, these are optional); in such cases, AnalysisTools names the bead types as `b0`, `b1`, etc., up to `bN` where `N` is the number of bead types and molecule types as `m0`, `m1`, etc. If unsure, the Info (section 3.8) utility can provide the names of beads and molecules for the specific structure file.

Info utility can also convert these formats between each other via the `-o <file>` option.

2.1.1 VTF format

See here for the complete description of `vtf/vsf` files (a `vtf` file contains a structure part followed by a coordinate part, while a `vsf` file contains only the structure part).

In this format, every bead is defined on its own line (except for the `atom default` bead type) and all bonds all listed.

Not all the keywords listed on the web page for the 'Atom lines' are recognized; keyword `n[ame]` is mandatory and optional keywords are `m[ass]`, `charge|q`, `r[adius]`, `resid`, and `res[name]`. For the `<aid-specifier>`, only a single bead index number or the `default` keyword can be used. Other keywords are ignored.

AnalysisTools groups beads (and molecules) into bead (and molecule) types. Generally, only name defines types of beads, i.e., should two beads share the name, they will be of the same type; their mass, charge, and radius will each equal to that for the bead type's topmost atom line containing the corresponding keyword. If the keyword is missing from all atom lines, that characteristic is undefined. In the case of the Info (section 3.8) utility (see there for details), the `--detailed` command line option triggers the detailed recognition, where beads that share the name but differ in mass, charge, and/or radius will be of different type (with a number appended to the original name).

Different molecule types, on the other hand, are always distinguished based on all their characteristics, i.e., name, connectivity, and order of bead types (order of its `vsf` indices). Molecule types may also be affected by the `--detailed` option because of the bead type definition.

For bond lines, only one bond per line is permitted (i.e., `bond <index1>:<index2>` format).

Periodic boundary condition can be included as `pbc <x> <y> <z>` for a cuboid box or `pbc <a> <b> <c> <alpha> <beta> <gamma>` for a general rhomboid box.

2.1.2 FIELD format

A format used by the DL_MESO_DPD simulation package concisely defines species of beads and molecules, providing number of beads/molecules of given species as well as their properties.

The first line is a header. If it begins with three numbers, they are the box sidelengths; three further numbers after them are the α, β, and γ cell angles in degrees, making the box triclinic. Anything after the numbers is free text, and a line that does not begin with three numbers is taken as a title, leaving the file without a box. The angles are written back out only when the cell is actually tilted, so an orthogonal box stays a plain three numbers. A `FIELD` file that defines no box can only be used where the box comes from elsewhere—when building a system from scratch with AddToSystem (section 3.1), for instance, you then have to supply it via `-b`.

The `species` section is the same as in the `DL_MESO FIELD` file, but the `molecules` section differs slightly. Besides the `beads` part, every molecule can have `bonds`, `angles`, and `dihedrals`, but even though up to three parameters for potential parameters are read, AnalysisTools does not recognize the potential keywords (`harm`, `cos`, etc.). Moreover, these potential parameters are only used for generating new `LAMMPS` or `DL_MESO` data files via, e.g., the Info (section 3.8) utility. The `molecules` section can also contain one extra part: `impropers` for improper angles which has the same format as the `dihedrals` section (this was added because the LAMMPS molecular

dynamics simulator and other software distinguish between the two types).

Note that the `FIELD` file does not have to contain the `Interactions` section; this part is ignored.

However, using this file in conjunction with coordinate files is not recommended. Because `FIELD` contains only numbers of beads of given types, it is not possible to ensure that bead indices in the coordinate file will line up with the correct bead types in the `FIELD` file. AnalysisTools assigns the bead indices on the 'first read, first labelled' basis, that is, the unbonded beads are first (in the order of the lines in the `species` section) followed by beads from the molecules. Should one want to check the bead index assignments, the `Info` utility can be used to generate, e.g., a `vsf` file from the `FIELD` file.

Nevertheless, the file can be used to supply extra information for bead and molecule types for the `Info` utility's `-i` option as these do not depend on individual beads but rather on bead and molecule type names (see Info (section 3.8) for details). Moreover, `FIELD` is used as an input file to add extra species into an existing system or to create a new system from scratch via, e.g., AddToSystem (section 3.1).

To be recognized by AnalysisTools utilities, the file must either be called `FIELD` or have the `.field` extension (case-insensitive); alternatively, the type can be specified explicitly with `-i <file> field`.

2.1.3 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description. An `xyz` file is essentially a file with timesteps (bead coordinates), and the only structure information are bead names. Note that the second line of each timestep (a comment line) is ignored.

When used as a structure file, the first timestep is used to define the system composition. Therefore while the timesteps can each have a different number of beads, their amount cannot exceed the number in the first step.

2.1.4 LAMMPS data format

This rather complicated format comes from the LAMMPS molecular dynamics simulator and lists the numbers of beads, bonds, angles, etc., as well as individual beads, bonds, angles, etc., and information about potentials for bonds, angles, etc. See here for description of the format and the `Examples` directory for example files.

AnalysisTools reads the header information as well as `Masses`, `Atoms`, `Velocities`, `Bonds`, `Angles`, `Dihedrals`, `Impropers`. It also reads `Bond Coeffs`, `Angle Coeffs`, `Dihedral Coeffs`, and `Improper Coeffs` sections.

The `Atoms` section may have `atom_style full`, `bond`, `angle`, `atomic`, `molecular`, or `charge` format (specified as a trailing comment on the `Atoms` line as required by the `LAMMPS` software).

For the `Bond Coeffs` and `Angle Coeffs` sections, harmonic potential is assumed, and the first value of each bond/angle type is multiplied by 2 because LAMMPS uses harmonic spring strength of $k/2$ (as opposed to, e.g., DL_MESO which uses k). For the `Dihedral Coeffs` and `Improper Coeffs` sections, AnalysisTools reads at

most three numbers without assuming any specific potential. The `Coeffs` sections are used only for generating new structure files via, e.g., the Info (section 3.8) utility; note that to run `LAMMPS` simulations, these section may have to be manually corrected to conform to the specific potential type.

To be recognized by AnalysisTools utilities, the file must have the `.data` extension; alternatively, use `-i <file> data` for a structure file or `-ft data` for a coordinate file.

2.1.5 LAMMPS lammpstrj format

This file format comes from the `LAMMPS` dump style custom command.

Of the possible `LAMMPS` attributes, AnalysisTools recognizes `id` (bead index – mandatory), `element` (bead name), `x y z` (Cartesian bead coordinates), `xs ys zs` and `xsu ysu zsu` (scaled coordinates, automatically converted to Cartesian), `xu yu zu` (unwrapped Cartesian coordinates), `vx vy vz` (bead velocities), `fx fy fz` (bead forces), `type` (bead type used as a fallback name when `element` is absent), and `q` (bead charge). Other attributes are ignored. Note that if `element` is missing, the `type` attribute is used as the bead name; if both are absent, all beads are considered of the same type called `b0`.

When used as a structure file, the first timestep is used to define the system composition. Therefore while the timesteps can each have a different number of beads, their amount cannot exceed the number in the first step.

To be recognized by AnalysisTools utilities, the file must have the `.lammpstrj` extension; alternatively, use `-i <file> ltrj` (or `lammpstrj`) for a structure file or `-ft ltrj` for a coordinate file.

2.1.6 DL_MESO CONFIG format

A coordinate file format used by the DL_MESO_DPD simulation package. The file starts with a title line, a line with two integers (key and imcon, where imcon specifies the periodic boundary condition type), and three lines defining the simulation box vectors. Each subsequent bead entry consists of a name-and-index line followed by a line with Cartesian coordinates.

Currently, AnalysisTools can write `CONFIG` files but not read them; it is therefore available as an output format only.

To be recognized as an output format, the file must be named `CONFIG` or have the `.config` extension (case-insensitive).

2.1.7 GROMACS ITP format

A topology file format used by the GROMACS molecular dynamics software. AnalysisTools reads the `[ moleculetype ]`, `[ atoms ]`, `[ bonds ]`, `[ angles ]`, and `[ dihedrals ]` sections. As `itp` files contain no coordinates, this format can only be used as a structure file.

To be recognized by AnalysisTools utilities, the file must have the `.itp` extension; alternatively, the type can be specified with `-i <file> itp`.

2.1.8 PDB format

The Protein Data Bank format is widely used by molecular simulation and visualisation programs. AnalysisTools reads the `CRYST*` record for the box dimensions (only the first three values, i.e. the box edge lengths; an orthogonal box is assumed) and `ATOM` records for bead information: the atom name field gives the bead type, the residue name field gives the molecule type, and the coordinate fields give the bead position. `HETATM` and other records are ignored. As `pdb` files contain no topology, this format can only be used as a structure file.

To be recognized by AnalysisTools utilities, the file must have the `.pdb` extension; alternatively, the type can be specified with `-i <file> pdb`.

2.2 Coordinate files

As a coordinate file, any of the following formats (described above) can be used:

- `vtf/vcf` format (`vtf` file contains both structure and coordinate sections while `vcf` file contains only the coordinate section); both indexed and ordered timesteps can be used, but the AnalysisTools utilities always output files with indexed timesteps
- `xyz` format where timesteps can have a different number of beads
- LAMMPS `data` format which by definition can only contain one timestep
- LAMMPS `lammpstrj` format where timesteps can have a different number of beads
- DL_MESO `CONFIG` format; currently available for output only (reading is not yet supported)

2.3 Aggregate file

An `agg` is generated using any of the `Aggregates` utilities. The file contains information about the number of aggregates in each timestep and which molecules belong to which aggregate. It serves as an additional input file for utilities that calculate aggregate properties; `agg` file is, therefore, linked to the structure and coordinate file(s) used to generate it.

The `agg` file is a simple text file. The first two lines are just comments (the second one should contain the command used to generate the file as parts of the command may be used by subsequent aggregate analysis). Starting at the third line, the data for individual timesteps are shown. It follows these rules:

- each timestep starts with `Step: <int>`
- the second line is the number of aggregates in the given timestep
- for each aggregate, there are *two* consecutive lines:
  - `<nCore> : <id1> <id2> ...` — the number of core molecules followed by their molecular indices (from the input structure file)
  - `<nBorder> : <id1> <id2> ...` — the number of border molecules followed by their molecular indices

  Core and border molecules are the result of the DBSCAN algorithm used to identify aggregates; free molecules (monomers) appear as singletons with one core molecule and no border molecules

- no blank or comment lines are allowed
- not all molecules present in the coordinate/structure file(s) used to generate this file must be present in every timestep
- the line `Last Step: <int>` signalizes the end of data (only the first word `Last` is checked)

Note that the term aggregate also refers to free molecules (i.e., unimers or fully dissolved chains).

If `vtf` format is used for the coordinates, the indices from `agg` file can be used in vmd to visualize, e.g., only a specific aggregate by using `resid <id1> ... <id\#>` in the `Selected Atoms` box inside the vmd. A rough script to visualize aggregates in different colours is in the `Examples/scripts/Visualize` directory (`VisAgg*` files), accompanied by the resultant picture (the bash script uses the provided `vtf` and `agg` files to generate a `tcl` script and run it via `vmd`).

An example of an `agg` file can be found in the `Examples/AggNumber` directory.

2.4 System info file

A system info file maps names to the molecule types present in a system. It is used with the `-sys` option in utilities such as AddToSystem (section 3.1) and Info (section 3.8) when molecule names are not available from the coordinate file.

The file is plain text with one line per molecule type (in the order they appear in the system), each formatted as a name followed by an integer count. Blank lines and comment lines (beginning with `#`) are ignored. The count value is not used when reading — it is only required for the line to be recognised — but when written by a utility, names are left-padded to 20 characters and counts to 7 digits. A warning is issued if the number of valid entries does not match the number of molecule types in the system.

2.5 Library directory

A library directory is specified via the `-lib` option (e.g. in AddToSystem (section 3.1) and Info (section 3.8)) and provides pre-defined bead types, interaction parameters, and molecule templates. It contains four index files and one file per molecule; there is no master list of molecules—a molecule exists if `<molname>.txt` is in the directory. An example library is in `Examples/library`.

Common grammar

All library files share the same grammar:

- `#` starts a comment that runs to the end of the line; blank lines are ignored.
- Before the first block, a line is a scalar: a key followed by its value(s), e.g. `role bilayer`.
- A line holding a single field is a block header, and every following row belongs to that block until the next header.
- A row inside a block is a row index followed by the block's columns.

The leading row index is decoration—the position in the block *is* the index and nothing reads the printed one—so it can silently drift out of step; `Info --check-library` (Info (section 3.8)) is there to catch that, along with dangling counterions, unknown type IDs, and molecules that do not balance.

Index files

- `list_parameters.txt` — one `bead_types` block of rows `<idx> <bead_ID> <mass> <q> <A> <rc> [source ...]`, where `A` is the DPD repulsion parameter, `rc` the cutoff radius (also taken as the bead radius), and the trailing source column is free text for people.
- `list_bonds.txt` — one `bond_types` block of rows `<idx> <bond_ID> <k_bond> <r0>` for the harmonic bond $U = k_{\mathrm{bond}}(r - r_0)^2/2$.
- `list_angles.txt` — one `angle_types` block of rows `<idx> <angle_ID> <k_angle> <theta0>` for the harmonic angle $U = k_{\mathrm{angle}}(\theta - \theta_0)^2/2$, with θ_0 in degrees.
- `list_cross_interactions.txt` — one `interactions` block of rows `<idx> <bead_i> <bead_j> <A_ij> <rc_ij> [source ...]`, giving the DPD parameters for a pair of different bead types.

The `bond_ID` and `angle_ID` strings are how molecule files refer to bond and angle types; the `bead_ID` is the bead type name that ends up in the system. A pair of different bead types with no row in `list_cross_interactions.txt` falls back to $A = 25$ and $r_c = 1$, not to the two types' own values, so an omitted pair is not the same as a 'sensible average'. The dissipative parameter γ is not read from the library at all—it is always 4.5.

Molecule files

Each molecule lives in its own `<molname>.txt`, loaded by name via `-mol` (or `-ntot`, or as another molecule's counterion). The scalars come first, then the `beads` block and the optional `bonds` and `angles` blocks, as in this shortened `CTAC.txt` from the example library:

```
name        CTAC
reference   from Prof. ML model
role        bilayer
M_w         319.99
counterion  Cl

beads
#   bead_ID                    x        y        z
 1  (CH3)3N+CH2           0.0000   0.0000   0.0000
 2  CH2CH2                0.0000   0.0000  -0.4900
 3  CH3                   0.0000   0.0000  -0.8800
```

```
bonds
#   bond_ID      i   j
 1  b04          1   2
 2  b02          2   3

angles
#   angle_ID     i   j   k
 1  a01          1   2   3
```

- `name` — the molecule type name; it should match the filename, since that is what the loader looks up.
- `role` — `bilayer` or `soluble`; required.
- `M_w` — molar mass, the mass you would weigh out (including the counterions). Optional, and its absence is meaningful: a species with no `M_w` is one you would not put in a recipe on its own.
- `counterion <name> [count]` — the molecule's counterion, with the count defaulting to 1. Up to four such lines are allowed.
- `label`, `reference` — free text for people. Any other scalar key is ignored, so the format can grow without old binaries refusing to read new files.

Rows in the `beads` block are `<idx> <bead_ID> <x> <y> <z>`, in `bonds <idx> <bond_ID> <i> <j>`, and in `angles <idx> <angle_ID> <i> <j> <k>`; the bead indices `i`, `j`, `k` are 1-based and must lie inside the `beads` block. Coordinates are relative to whatever the placing utility uses as the molecule's reference point. A block name other than these three is an error, as is a missing `role` line or a missing `beads` block. A molecule is limited to 64 beads and to 256 bonds and angles; the library as a whole to 64 bead, bond, and angle types.

A one-bead molecule (a solvent bead, a bare ion) becomes an unbonded bead in the built system rather than a molecule type, so it needs no bonds and gets no molecule type name.

Counterions

A counterion is a molecule in its own right, named by a `counterion` line and stored in its own file—never beads appended to the parent. Adding n copies of a molecule therefore also adds $n \times$ count copies of each declared counterion, which for a monoatomic ion means free beads floating in the box (use `--no-cion` in AddToSystem (section 3.1) to place them yourself).

Resolution is exactly one level deep: a counterion contributes its beads and its charge and nothing else. Its own `counterion` line is not followed and its `M_w` is not consulted, because the parent's `M_w` already covers the whole salt. A file can be both—`Na.txt` can be a listed salt with a counterion of its own *and* somebody else's counterion—and only the role it is being used in decides how it is read.

3. Utilities

All utilities have command line options affecting their behaviour. The following options are common for many utilities, so they are described here rather than at the individual utilities where only list of the possible options is provided.

General options	
`--verbose`	print system composition (bead and molecule types, counts, bond/angle/dihedral types, and box dimensions) to the screen
`--silent`	suppress all progress and informational output (overrides `--verbose`; errors and warnings are still shown)
`--help`	print short description of the utility and exit
`--version`	print version of the utilities and exit
Options for structure and coordinate input files	
`-i <stru> [type]`	use specified structure file; optional `type` overrides extension-based format detection, see Structure files (section 2.1) for possible formats and type strings
`-ft <type>`	override input file type: `vtf`, `xyz`, `data`, `ltrj` (or `lammpstrj`), `field`, `itp`, `pdb`
`-st <n>`	start calculations at `n`-th timestep
`-e <n>`	end calculations at `n`-th timestep
`-sk <n>`	skip every `n` timesteps (i.e., use every `(n+1)`-th timestep)

The `-st`, `-e`, and `-sk` options can be combined, so that, for example, specifying `-st 2 -e 10 -sk 2` would use timesteps 2, 5, and 8.

Should two (or more) identical options be specified on the command line, only the first one is used.

Note that while in theory it is possible to combine any coordinate and structure file formats via the `-i` option, it is generally not recommended. For example, `xyz` file format does not include bead indices which means that beads are numbered according to the file line which may not agree with `vsf` structure format that does contain bead indices.

3.1 AddToSystem

`AddToSystem` adds unbonded beads and/or molecules to an existing system, or builds a new system from scratch. New species can be placed randomly or subject to distance and axis constraints; they can either replace existing beads or be appended

to the system.

3.1.1 Defining what to add

Species to add are specified in one of two ways. The simpler is a `FIELD` file (`<in.field>`) given as the second positional argument, which is read as-is.

The library-based workflow uses `-lib <dir>` to point to a molecule library (Library directory (section 2.5)) and `-mol` to select which species to include. After `-mol`, each entry is a name–count pair: the count is either an integer number of molecules or a percentage of the original system's bead count (e.g. `10\%` gives round$(0.1 \times N_{\text{orig}}/n_{\text{beads}})$ molecules, where n_{beads} is the number of beads per molecule). Multiple name–count pairs can follow a single `-mol` flag, and the flag can be repeated. Note that percentage counts are meaningless when building from scratch because the original bead count is zero; use integer counts in that case.

The `-ntot <n> <type>` option (requires `-lib`) adds fill molecules of type `<type>` until the total bead count reaches `n`. The fill count is computed via integer division: $\lfloor$(target−current−-mol beads)$/n_{\text{fill}}\rfloor$, where n_{fill} is the bead count of one fill molecule. When building from scratch, the current count is taken as zero.

Counterions. A library molecule can declare counterions, and they come along automatically: adding n copies of `CTAC` also adds n chlorides, as separate molecules (free beads, for a monoatomic ion) rather than as extra beads of the parent. They are placed like anything else, subject to the same constraints. Both n_{beads} above and n_{fill} are the bead counts *including* the declared counterions, so `-mol CTAC 1\%` of a 12 000-bead system gives $0.01 \times 12\,000/(9+1) = 12$ molecules, i.e. 12 `CTAC` plus 12 `Cl-` beads.

Use `--no-cion` to add only the named molecules and place their counterions yourself. This is what the bilayer workflow needs: a counterion belongs in the water, not in the leaflet its parent is being constrained into, so you add the surfactants with the axis constraints and then add the ions in a second run without them. Note that the counts are computed the same way either way: `--no-cion` suppresses the counterions but does not remove their beads from the n_{beads} used for percentages and for `-ntot`, so a percentage count still comes out as though they were there.

When `-lib` is used with an existing system, `-sys <input>` is also required; it reads a system-info file that maps the existing molecule types to library names, keeping bead-type names consistent. An updated system-info file can be written by supplying a second filename to `-sys`. When building from scratch, a single `-sys` argument is treated as output-only.

3.1.2 Placement constraints

By default, each new bead—or the reference point of each new molecule (the geometric centre by default; see section 3.1.3)—is placed uniformly at random within the simulation box. Two families of constraint can narrow this region.

Distance constraints. `-ld <d>` and `-hd <d>` require the distance from the nearest reference bead to be, respectively, strictly greater than `d` or strictly less than `d`; distances are computed with periodic boundary conditions. Reference beads are selected by type (`-bt <name(s)>`) or taken as all bonded beads in the system (`--bonded`, which overrides `-bt`); at least one of these is required when either distance constraint is active. The two can be combined to restrict placement to a shell at distance (ld, hd) from the reference beads.

When `-hd` is used, the candidate region is pre-narrowed to the bounding box of the reference beads expanded by the `-hd` value in every direction, which speeds up placement when the reference beads occupy only a small part of the box.

Axis constraints. `-cx <lo> <hi>` (and `-cy`, `-cz`) restrict the corresponding coordinate of the reference point to the interval $[lo, hi)$. By default the bounds are fractions of the output box sidelength (values in $[0, 1]$); `--real` switches to absolute coordinates. If `lo > hi`, the values are swapped silently. For a triclinic box the bounds mean something slightly different—see section 3.1.6.

Each option takes up to ten `<lo> <hi>` pairs, and the allowed region for that axis is their union; the pairs need not be given in order, and overlapping ones—or ones that merely touch—are merged, with a warning. A range is drawn with a probability proportional to its width, so the density stays uniform over the whole union. The axes are independent— `-cx` can take three pairs while `-cy` takes one—and the region is the product of the per-axis unions. For instance,

```
-cx 0 0.2 0.8 1.0 -cy 0 0.5
```

gives two slabs at opposite ends of the x-axis, both cut down to the lower half of the box in y.

Distance and axis constraints can be freely combined. figure 3.1 shows examples. `AddToSystem` retries placement until all constraints are satisfied; if no valid position is found within 10 000 000 attempts it exits with an error. No feasibility check is performed in advance, so an unsatisfiable combination is only detected after all retries are exhausted.

3.1.3 Molecular orientation

New molecules are randomly rotated by default, using a ZYZ Euler-angle scheme that produces a uniform distribution over all orientations. Two options change this: `--no-rotate` preserves the orientation from the `FIELD` file, and `-a <$\alpha $> <$\beta $> <$\gamma $>` applies a fixed rotation by α, β, and γ degrees around the x-, y-, and z-axes, respectively (overrides `--no-rotate`). The angles follow the right-hand rule, and the rotations are composed as $R = R_z(\gamma)R_y(\beta)R_x(\alpha)$, i.e. the x-rotation is applied to the molecule first:

$$R = \begin{bmatrix} \cos\gamma\cos\beta & \cos\gamma\sin\beta\sin\alpha - \sin\gamma\cos\alpha & \cos\gamma\sin\beta\cos\alpha + \sin\gamma\sin\alpha \\ \sin\gamma\cos\beta & \sin\gamma\sin\beta\sin\alpha + \cos\gamma\cos\alpha & \sin\gamma\sin\beta\cos\alpha - \cos\gamma\sin\alpha \\ -\sin\beta & \cos\beta\sin\alpha & \cos\beta\cos\alpha \end{bmatrix}. \tag{3.1}$$

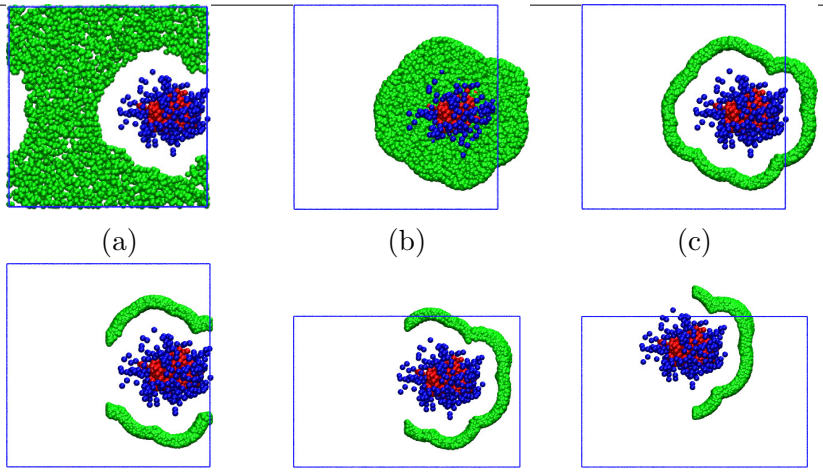


Figure 3.1: Placement constraint examples: (a) `-ld` only, (b) `-hd` only, (c) both `-ld` and `-hd`, (d) adding an `-cx` axis constraint, (e) further adding `-b` to redefine the simulation box, and (f) adding `-off` to shift the original system. Red and blue: original beads; green: added beads; blue line: new simulation box. The pictures show cross-sections of the box. See `Examples/AddToSystem/01-ManualExamples` for the exact commands.

Placement constraints are applied to the molecule's *reference point*, which is the geometric centre by default; `--head` switches this to the first bead, and `--tail` to the last bead (`--head` takes priority if both are given). The molecule is rotated around the reference point, which is then placed at the constrained position. The remaining beads are positioned relative to it using the rotated `FIELD` coordinates, so they are not themselves guaranteed to satisfy any constraints.

3.1.4 Exchange and append mode

When adding to an existing system, `AddToSystem` by default *exchanges* beads: for each new bead placed, one existing bead is removed, keeping the total bead count approximately constant. The type to remove is the most numerous bead type unless `-xb <type>` specifies otherwise. No check is done whether the removed bead belongs to a molecule; removing a bonded bead will leave an incomplete molecule in the output.

Using `--add` appends new species without removing any existing beads, increasing the total bead count (this overrides `-xb`). When building from scratch, append mode is always active regardless of `--add`.

3.1.5 Box, offset, and scratch mode

`-b <x> <y> <z>` sets the output simulation box sidelengths (always in absolute units, regardless of `--real`). Three further numbers, `-b <x> <y> <z> <$\alpha $> <$\beta $> <$\gamma $>`, add the cell angles in degrees and make the output box triclinic; without them the box is orthogonal. Each angle must lie in $(0, 180)$ and

the six numbers together must describe a realisable cell, otherwise `AddToSystem` exits with an error. By default the new box is centred on the original one. All placement constraints refer to the output box, not the input one.

`-off <x> <y> <z>` shifts all beads from the input system before the new species are placed. Without `--real`, the amounts are fractions of the cell vectors (which for an orthogonal box are simply fractions of the sidelengths); with `--real` they are in absolute units. Periodic boundary conditions are not applied after the shift, so beads can move outside the box.

To create a system from scratch, pass `-` in place of the input file; the box is then taken from the `FIELD` file or library, including its angles if the first line of the `FIELD` file lists six numbers. In this mode, `-ld`, `-hd`, `-bt`, `--bonded`, `-xb`, and `-st` have no effect and a warning is printed if any of them are supplied.

An extra output file in any supported format can be written with `-o <file>`, which is useful when both a `vtf` visualisation file and a `data` LAMMPS file are needed from a single run. Use `-ebt <int>` to reserve extra bead-type slots in a LAMMPS `data` output. The random seed can be fixed with `-s <int>` for reproducibility.

3.1.6 Triclinic boxes

Triclinic cells are supported throughout: beads and molecules are placed uniformly inside the tilted cell, and the `-ld`/`-hd` distance checks use the minimum-image convention appropriate for it. A triclinic output box comes either from the input file, from a `FIELD` file whose first line carries the three angles, or from `-b` with six numbers.

The axis constraints need more care. A slab bounded by two planes perpendicular to a Cartesian axis cannot be periodic in a cell tilted in that plane, so there is no Cartesian sub-box to place beads in. For a triclinic box, `-cx`, `-cy`, and `-cz` are therefore interpreted as fractions along the first, second, and third *cell vector*, carving out a smaller cell of the same shape. The constrained region then shears with the cell, so its periodic images line up as they should. For an orthogonal box the two readings coincide and nothing changes.

Because of this, `--real` cannot be combined with `-cx`/`-cy`/`-cz` when the box is triclinic; `AddToSystem` exits with an error asking you to use fractions instead. The `-hd` option is unaffected: the region it derives is a bounding box around existing beads, which is axis-aligned regardless of the cell shape.

Further examples—including incremental builds, asymmetric bilayer assembly, cylindrical aggregates, and library-based workflows—are in `Examples/AddToSystem`.

Usage: `AddToSystem <input> [<in.field>] <output> [options]`

Mandatory arguments	
`<input>` / `-`	input coordinate file, or `-` to create a system from scratch

`[<in.field>]`	`FIELD` file defining species to add (omit when `-lib` is used)
`<output>`	output coordinate file
Options	
`-o <file>`	write an extra output file in any supported format
`-lib <dir>`	molecule library directory; use with `-mol` instead of `<in.field>`
`-mol <mol> <\%/n> ...`	molecule name(s) from the library with percentage or integer count; pairs can be repeated
`-sys <input> [output]`	system-info file mapping existing molecule types to library names; optionally write updated info (with `-`, a single argument is output-only)
`-ntot <n> <type>`	fill to `n` total beads using `<type>` from the library (requires `-lib`)
`--no-cion`	add only the named molecules, not the counterions they declare (place those yourself)
`-ld <float>`	minimum distance from reference beads (exclusive)
`-hd <float>`	maximum distance from reference beads (exclusive)
`-bt <name(s)>`	bead types to use as reference for `-ld`/`-hd`
`--bonded`	use all bonded beads as reference for `-ld`/`-hd` (overrides `-bt`)
`-xb <type>`	bead type to exchange (default: most numerous type)
`--add`	append new species without exchanging existing beads (overrides `-xb`)
`--no-rotate`	do not randomly rotate added molecules
`-a 3x<angle>`	rotate all added molecules by given angles around x, y, z axes in degrees (overrides `--no-rotate`)
`--head`	use molecule's first bead as constraint reference point (overrides `--tail`)
`--tail`	use molecule's last bead as constraint reference point (default: geometric centre)
`-cx <lo> <hi> ...`	constrain x coordinate to $[lo, hi)$ (first cell vector if the box is triclinic); up to ten pairs, giving a union of ranges
`-cy <lo> <hi> ...`	constrain y coordinate to $[lo, hi)$ (second cell vector if the box is triclinic); up to ten pairs, giving a union of ranges
`-cz <lo> <hi> ...`	constrain z coordinate to $[lo, hi)$ (third cell vector if the box is triclinic); up to ten pairs, giving a union of ranges
`--real`	use absolute units for `-cx`/`-cy`/ `-cz`/`-off` (default: fraction of the output box); not allowed with `-cx`/`-cy`/`-cz` for a triclinic box

`-b <x> <y> <z>`	output simulation box sidelengths (absolute units), optionally followed by the α, β, γ cell angles in degrees for a triclinic box
`-off <x> <y> <z>`	shift input system beads (fraction of the cell vectors, or absolute with `--real`)
`-s <int>`	seed for the random number generator
`-ebt <int>`	reserve extra bead-type slots in LAMMPS `data` output
`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

TODO: Distance checking against the first bead of each molecule (mode 3 in the source code) is mentioned in a comment but appears to not be implemented.

3.2 AggNumber

`AggNumber` calculates the time evolution of average aggregate sizes, distributions of aggregate sizes, and/or composition distributions of aggregates of specified sizes from an `agg` file (see Section 2.3). At least one of the `-a`, `-d`, and `-c` options must be specified.

For a quantity $\mathcal{O}$ measured per aggregate, its number, weight, and z averages are

$$\langle \mathcal{O} \rangle_{\mathrm{n}} = \frac{\sum_i N_i \mathcal{O}_i}{N}, \qquad \langle \mathcal{O} \rangle_{\mathrm{w}} = \frac{\sum_i N_i m_i \mathcal{O}_i}{\sum_i N_i m_i}, \qquad \langle \mathcal{O} \rangle_{\mathrm{z}} = \frac{\sum_i N_i m_i^2 \mathcal{O}_i}{\sum_i N_i m_i^2}, \tag{3.2}$$

where N is the total number of measured aggregates, N_i is their number with value $\mathcal{O}_i$, and m_i is the mass of aggregate i.

The number, weight, and z distribution functions of aggregate sizes are

$$F_{\mathrm{n}}(A_{\mathrm{S}}) = \frac{N_{A_{\mathrm{S}}}}{N}, \qquad F_{\mathrm{w}}(A_{\mathrm{S}}) = \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}}{M}, \qquad F_{\mathrm{z}}(A_{\mathrm{S}}) = \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}^2}{\sum_{A_{\mathrm{S}}} N_i m_i^2}, \tag{3.3}$$

where $N_{A_{\mathrm{S}}}$ and $m_{A_{\mathrm{S}}}$ are the count and mass of aggregates with size A_{S}, M is the total mass of all measured aggregates, and the distributions are normalised to $\sum F_x(A_{\mathrm{S}}) = 1$.

3.2.1 Aggregate size

By default, the aggregate size A_{S} is the total number of molecules in the aggregate. The `-m`, `-x`, and `-only` options alter both which aggregates are considered and how their size is defined:

- `-m <name(s)>`: A_{S} counts only the specified molecule types; aggregates where this count is zero are skipped.
- `-x <name(s)>`: aggregates composed exclusively of the specified molecule type(s) are excluded.

- `-only <name(s)>`: only aggregates whose molecules all belong to the specified type(s) are included.

These can be combined freely. For example, `-only A B -x A` selects aggregates composed only of `A` and `B` molecules but further excludes those containing only `A`. The `-n <int> <int>` option restricts the analysis to a given range of A_S as defined by the options above.

3.2.2 Mass definition

When the `-m` option is used, aggregate mass can be defined as either the 'partial mass' (mass of the `-m`-selected molecule types only) or the 'total mass' (mass of all molecules in the aggregate). Both give physically meaningful weight and z averages of aggregate size and size distributions, so the `-a` and `-d` output files provide both. For all other quantities (average aggregate mass $\langle M \rangle$, shape descriptors from `GyrationAggregates`, etc.) only total mass is physically meaningful, so only total-mass-based values are written. When `-m` is not used, partial and total mass are identical and only one set of values is needed; the output files provide two identical columns for easier parsing.

TODO: Add AggNumber examples to `Examples/`.

TODO: Warning for zero-mas aggregates that produce nan in output is not yet implemented.

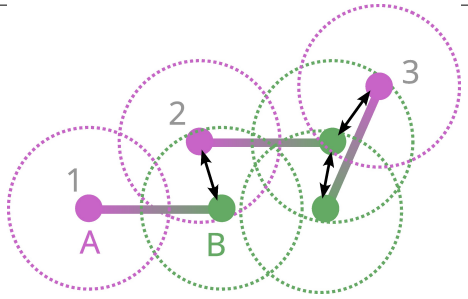
Usage: `AggNumber <in.stru> <in.agg> [options]`

Mandatory arguments	
`<in.stru>`	input structure file
`<in.agg>`	input `agg` file
Output options (at least one required)	
`-a <file>`	write per-timestep averages
`-d <file>`	write distribution of aggregate sizes
`-c <file> <size(s)>`	write composition distributions for the specified aggregate size(s); creates two files per size: `<file>-<NNN>.txt` and `<file>_r-<NNN>.txt`, where `<NNN>` is the size zero-padded to three digits
Aggregate size options	
`-m <name(s)>`	use the count of the specified molecule type(s) as A_S (Section 3.2.1)
`-x <name(s)>`	exclude aggregates composed only of the specified molecule type(s)
`-only <name(s)>`	include only aggregates composed entirely of the specified molecule type(s)
`-n <int> <int>`	restrict to aggregates with A_S in a given range
Other options (see the beginning of Chapter 3)	

`-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`

Format of output files:

1) `-a <file>` – per-timestep averages
   - first two lines: AnalysisTools version and the command used
   - third line: column headers
     - `step`: simulation timestep
     - `<As>_n`: number-average aggregate size
     - `<As>_w`, `<As>_z`: weight and z averages of A_S (Eq. (3.2)); two columns each, one using partial mass and one using total mass (Section 3.2.2)
     - `<M>_n`, `<M>_w`, `<M>_z`: number, weight, and z averages of aggregate mass (total mass only; Eq. (3.2))
     - `<name>_n` for each molecule type: average count of that type per aggregate; summing over all types gives `<As>_n`
     - `n_agg`: number of eligible aggregates in the timestep
   - fourth line: note explaining the partial/total mass distinction
   - data lines follow; all values are zero in timesteps where no eligible aggregates exist
   - last two lines: overall averages across the entire run, commented with `\#`; same quantities as the per-timestep columns except `n_agg` is replaced by `<n_agg>` (mean number of aggregates per timestep)
2) `-d <file>` – distribution of aggregate sizes
   - first two lines: AnalysisTools version and the command used
   - third line: column headers
     - `As`: aggregate size
     - `F_n(As)`, `F_w(As)`, `F_z(As)`: number, weight, and z distribution functions (Eq. (3.3)); two columns for `F_w(As)` and `F_z(As)`, one per mass definition
     - `n_agg`: total count of aggregates of this size across all timesteps
     - `<name>_n` for each molecule type: average count of that type in an aggregate of this size
   - fourth line: note explaining the partial/total mass distinction
   - data lines follow; only sizes observed at least once are written
   - last two lines: same overall averages as in `-a`
3) `<file>-<NNN>.txt` from `-c` – per-type composition distribution
   - first two lines: AnalysisTools version and the command used
   - third line: total number of aggregates of the given size found across all timesteps
   - fourth line: column headers – (1) number of molecules of a given type (from 0 to A_S); one column per molecule type giving the fraction of aggregates containing exactly that many of that type
   - data lines follow; all molecule types present in the aggregate are counted, regardless of the `-m` option
4) `<file>_r-<NNN>.txt` from `-c` – pairwise composition distribution
   - first two lines: AnalysisTools version and the command used
   - third line: total number of aggregates of the given size
   - fourth line: column headers – (1–2) counts of two molecule types; one column per distinct molecule-type pair giving the fraction of aggregates with exactly



`Aggregates -bt A B`
⇒ 3 contact pairs

`Aggregates -bt A B --pairs`
⇒ 2 contact pairs

Figure 3.2: Contact-detection example: three two-bead molecules. Dotted lines indicate the maximum contact distance (`-d` option); arrows show detected contact pairs. The same bead can appear in multiple pairs.

those counts; pairs with no observations are printed as `?`

- data lines follow (only rows where at least one pair has a non-zero value); a blank line separates blocks where the first-column count changes, making the file directly usable as a gnuplot 3D dataset

3.3 Aggregates

`Aggregates` determines which molecules belong to which aggregate and writes the result to an `.agg` file (see Section 2.3). The `.agg` file is the required input for aggregate-analysis utilities such as AggNumber (section 3.2), ?? (??), and ?? (??).

3.3.1 Contact detection

Two beads from different molecules form a *contact pair* if their distance is at most `-d` (default: 1). Two molecules are *connected* if they share at least `-c` contact pairs (default: 1). Distances are calculated with periodic boundary conditions; use `--no_pbc` to ignore them.

By default all bead types and all possible bead type pairs are used. The `-bt <bead(s)>` option restricts detection to the listed types, checking all pairwise combinations among them (including same-type). For example, `-bt A B` checks A-A, A-B, and B-B contacts.

Adding `--pairs` changes the interpretation of `-bt`: the arguments are read as consecutive bead type pairs rather than a type list, so `-bt A B --pairs` checks only A-B contacts, and `-bt A B C D --pairs` checks A-B and C-D. The `--pairs` option requires the `-bt` one.

Figure 3.2 illustrates the difference on a small example.

3.3.2 Aggregate identification

Aggregates are found using DBSCAN at the molecule level: a molecule is a *core point* if it has at least `-nbr` connected neighbours (default: 1); a molecule with fewer neighbours that is nonetheless reachable from a core point is a *border point*;

all remaining molecules become singletons. The core/border split is recorded in the `.agg` file; singletons appear as a single core molecule with no border molecules.

The default `-nbr 1` makes every connected molecule a core point, so the algorithm reduces to simple connected components and no border molecules arise. Raising `-nbr` is useful when loose contacts should not extend clusters into spiderweb-like structures.

3.3.3 Additional output

Joined coordinates

The `-j <file>` option writes a coordinate file with PBC removed for aggregates: each aggregate's molecules are shifted so they appear connected across box boundaries, with the aggregate's centre of mass inside the simulation box. This is useful for visualization and avoids repeated PBC-removal for aggregates in subsequent analyses.

Wall separation

The `-w <a> <float(s)>` option separates wall-touching aggregates from bulk ones. `<a>` is the axis perpendicular to the wall(s) (`x`, `y`, or `z`); the floating-point argument(s) give the wall coordinate(s) along that axis. An aggregate is wall-touching if any of its `-bt` beads lies within the contact distance `-d` of a wall. For example, in a box of side 10 with slit walls in the xy-plane, `-w z 1 9` could be used.

Two extra `.agg` files are written alongside the main output: `<out>_w.agg` for wall-touching aggregates and `<out>_b.agg` for bulk aggregates (the main file still contains all aggregates). When `-j` is also used, matching joined-coordinate files with `_w`/`_b` suffixes are created as well.

Coupling between output files

The main `<out.agg>` is linked to the coordinate file used to generate it: most downstream utilities read both files together, so they must cover the same timesteps. When `-st`, `-e`, or `-sk` are used, the resulting `.agg` file is no longer tied to the original coordinate file but is coupled with the `-j` output (if given); only use the two coupled files for further analysis.

Usage: `Aggregates <coor> <out.agg> [options]`

Mandatory arguments	
`<coor>`	input coordinate file
`<out.agg>`	output `agg` file
Options	
`-bt <bead(s)>`	bead types for contact detection (default: all)
`--pairs`	treat `-bt` arguments as explicit bead type pairs; `-bt` is mandatory in this case

`-d <float>`	maximum contact distance (default: 1)
`-c <int>`	minimum contacts to connect two molecules (default: 1, max: 255)
`--no_pbc`	ignore periodic boundary conditions for contact check
`-nbr <int>`	DBSCAN minimum neighbours for a core molecule (default: 1)
`-j <file>`	write joined coordinates (PBC removed per aggregate)
`-w <a> <float(s)>`	split output by proximity to wall(s) at the given coordinate(s) on axis `<a>`
Other options (see the beginning of Chapter 3)	
`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

Output files:

1) `<out.agg>` – aggregate data for all aggregates in each timestep; format described in Section 2.3
2) `-j <file>` – coordinate file (format from extension) with PBC removed per aggregate; one timestep per input frame, matching `<out.agg>`
3) `-w <a> <float(s)>` – two additional `.agg` files: `<out>\_w.agg` for wall-touching aggregates and `<out>\_b.agg` for bulk aggregates; if `-j` is used, two matching joined-coordinate files are also written

TODO: Add Aggregates usage examples to `Examples/Aggregates`.

3.4 AngleMolecules

`AngleMolecules` calculates distributions of angles in specified molecule type(s), grouped by bead type triplet. It can also compute distributions for arbitrary bead trios via the `-n` option.

The angle θ at vertex bead j is the angle between the two bond vectors pointing away from it:

$$\theta_{ijk} = \arccos\left(\frac{(\mathbf{r}_i - \mathbf{r}_j)\cdot(\mathbf{r}_k - \mathbf{r}_j)}{|\mathbf{r}_i - \mathbf{r}_j|\,|\mathbf{r}_k - \mathbf{r}_j|}\right), \tag{3.4}$$

where i and k are the two outer beads. Angles are in degrees, ranging from 0 to 180.

The structure file must have angles defined (e.g. a LAMMPS `data` file). For each selected molecule type, angles are grouped by bead type triplet, labelled as `OuterType1-VertexType-OuterType2` where the two outer types are sorted by their internal type index. For example, a linear molecule with bead order A-A-A-B-A-A and angles 1-2-3, 2-3-4, 3-4-5, 4-5-6 has three distinct triplet types: A-A-A, A-A-B, and A-B-A. By default all molecule types are used; restrict this with `-m`.

Adding `--all` generates additional per-individual-angle distributions alongside the per-type results — angle 1, angle 2, etc., in the order they appear in the structure file. This is useful when structurally inequivalent angles share the same bead type triplet.

The `-n` option computes distributions for angles specified by bead index trios. Indices are 1-based in the order beads appear in the structure file (the `Info` utility

shows these). Trios are given as consecutive groups of three integers after the output filename, e.g. `-n angles.txt 1 3 5` computes the angle at bead 3 between beads 1 and 5. Multiple trios can be given at once. Unlike `BondLength`'s `-n`, there is no fallback for out-of-range indices: if any index in a trio exceeds the number of beads in a molecule, that trio is silently skipped for that molecule. Bead indices must always be supplied; providing `-n` without any integers is an error.

If the coordinate file contains molecules split across periodic boundaries, `AngleMolecules` joins them before the calculation. Provide `--joined` if the coordinates are already whole-molecule to skip this step.

`Usage: AngleMolecules <input> <width> <output> [options]`

Mandatory arguments	
`<input>`	input coordinate file (structure file must define angles)
`<width>`	bin width in degrees
`<output>`	output file with angle distributions

Options	
`-m <name(s)>`	molecule type(s) to use (default: all)
`--joined`	input already contains joined (whole-molecule) coordinates
`--all`	also write per-individual-angle distributions
`-n <file> <ints>`	write distributions for angles given as groups of three 1-based bead indices to `<file>`; indices must be provided

`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`

Format of output files:

1) `<output>` — angle distributions grouped by bead type triplet
   - 1st line: command used to generate the file
   - next lines (one per selected molecule type): comment lines listing column numbers and bead type triplet labels (`OuterType1-VertexType-OuterType2`); if `--all` is used, the column range for individual angles is noted at the end of each line
   - data lines: column 1 is the bin centre in degrees; remaining columns are normalised angle distributions
   - second-to-last line: comment line mapping column numbers to angle types, three columns per type in the order min, max, average
   - last line: commented-out minimum, maximum, and average angle for each type
2) `-n <file>` — distributions for angles specified by bead indices
   - 1st line: command used to generate the file
   - next lines (one per selected molecule type): comment lines listing column numbers and bead index trios (written as `i-j-k` with the outer indices sorted so $i < k$); trios where any index exceeds the molecule's bead count are omitted
   - data lines: same structure as `<output>`
   - second-to-last line: comment line mapping columns to bead index trios, three columns per trio in the order min, max, average
   - last line: commented-out minimum, maximum, and average values

TODO: Currently no error is raised when no angles are defined in the structure file and `-n` is not used; the utility will run but produce an empty output.

3.5 Average

This utility calculates simple average and standard error from specified column(s) of data in the input text file (all `\#`-starting lines and blank lines are ignored), printing it to standard output. It can also calculate one of three types of averages based on a used option – an overall average with statistical error and an autocorrelation time estimate (`-tau` option), block averages (`-b` option), or moving averages (`-m` option). While the `-tau` option appends a single line to the output file, either of the `-b` or `-m` options creates a new output file with somewhat smoother data.

The first and last lines used for the average calculation can be controlled using the standard `-st` and `-e` options.

3.5.1 Estimate autocorrelation time via -tau option

The average value of an observable $\mathcal{O}$ is a simple arithmetic mean,

$$\langle \mathcal{O} \rangle = \frac{1}{N} \sum_{i=1}^{N} \mathcal{O}_i, \tag{3.5}$$

where N is the number of measurements and the subscript i denotes individual measurements. If the measurements are independent (i.e., uncorrelated), the statistical error, ϵ, is given by:

$$\epsilon^2 = \frac{\sigma^2_{\mathcal{O}_i}}{N}, \tag{3.6}$$

where $\sigma^2_{\mathcal{O}_i}$ is the variance of the individual measurements,

$$\sigma^2_{\mathcal{O}_i} = \frac{1}{N-1} \sum_{i=1}^{N} (\mathcal{O}_i - \langle \mathcal{O} \rangle)^2. \tag{3.7}$$

For correlated data, the autocorrelation time, τ, representing the number of steps between two uncorrelated measurements must be determined. Every τ-th measurement is uncorrelated, so the equation (3.6) can then be used to estimate the error.

A commonly used method to estimate τ is the binning (or block) method. In this method, the correlated data are divided into N_B non-overlapping blocks of size k ($N = kN_\mathrm{B}$) with per-block averages, $\mathcal{O}_{\mathrm{B},n}$, defined as:

$$\mathcal{O}_{\mathrm{B},n} = \frac{1}{k} \sum_{\substack{i=1+ \\ (n-1)k}}^{kn} \mathcal{O}_i. \tag{3.8}$$

If $k \gg \tau$, the blocks are assumed to be uncorrelated and equation (3.6) can be used:

$$\epsilon^2 = \frac{\sigma^2_\mathrm{B}}{N_\mathrm{B}} = \frac{1}{N_\mathrm{B}(N_\mathrm{B}-1)} \sum_{n=1}^{N_\mathrm{B}} (\mathcal{O}_{\mathrm{B},n} - \overline{\mathcal{O}})^2. \tag{3.9}$$

An estimate of the autocorrelation time can be obtained using the following formula:

$$\tau_{\mathcal{O}} = \frac{k\sigma_{\mathrm{B}}^2}{2\sigma_{\mathcal{O}_i}^2}. \tag{3.10}$$

The number of blocks, N_{B}, is supplied as an argument of the `-tau` option and Average then appends a single line to the `<output>` file; the line starts with N_{B} and continues with three values ($\langle\mathcal{O}\rangle$, ϵ, and $\tau_{\mathcal{O}}$) per every data column specified in the Average command.

A way to quickly get a τ estimate is to use a wide range of N_{B} values and plot $\tau_{\mathcal{O}}$ from equation (3.10) as a function of N_{B}. Because the number of data points in one block should be significantly larger than the autocorrelation time (e.g., ten times larger), plotting $f(x) = N/(10x)$ will produce a monotonously decreasing curve that intersects the $\tau_{\mathcal{O}}$ vs. N_{B} curve. A value of $\tau_{\mathcal{O}}$ near the intersection (but to the left, where the decreasing curve is above $\tau_{\mathcal{O}}$ vs. N_{B} curve) can be considered a safe estimate of τ.

3.5.2 Block averages via -b option

Besides estimating the autocorrelation time, the per-block averages can be themselves plotted to get (probably) smoother dataset. Using the `-b` option, the number of data points per block to average, k, is supplied, and the utility prints the per-block averages from equation (3.8) to the output file.

This way of averaging could be useful for example with density data produced by DensityBox or related utilities; if the bin width supplied to the DensityBox was too small, it is (possibly much) faster to block-average the densities rather than rerun DensityBox. For this case, specify the first column (distance) along with any density columns from the density file.

3.5.3 Moving averages via -m option

More common way to smoothing noisy data is to use the moving (or rolling or running) average (or moving mean or rolling mean); this common method does have many names.

Similarly to the block-average, the first element of the moving average is a simple mean of k values. Unlike with block-average, however, the k values for the next element are obtained by ignoring only one value and taking the next k values; i.e., $k-1$ values from the previous element of the moving average are always reused as opposed to the block-average where the blocks of data are not overlapping.

The moving average elements, $\mathcal{O}_{\mathrm{M},n}$, are defined as

$$\mathcal{O}_{\mathrm{M},n} = \frac{1}{k}\sum_{n}^{n+k-1}\mathcal{O}_i. \tag{3.11}$$

Usage: `Average <input> <output> <column(s)> [options]`

Mandatory arguments	
`<input>`	input text file
`<output>`	output text file
`<column(s)>`	at least one column number from `<input>`

Options	
`-tau <int>`	τ estimation where `<int>` represents the number of blocks N_B
`-b <int>`	block-average printing mode where `<int>` represents the number of data points per one block, k (equation (3.8))
`-m <int>`	moving average mode where `<int>` represents the number of data points per one moving block, k (equation (3.11))

Other options (see the beginning of Chapter 3)
`-st`, `-e`, `--verbose`, `--silent`, `--help`, `--version`

3.6 BondLength

`BondLength` calculates distributions of bond lengths for specified molecule type(s), grouped by bead type pair. It can also calculate distributions of distances between arbitrary bead pairs within those molecules.

For each selected molecule type, bond lengths are accumulated into distributions per bead type pair. For example, a molecule A-B-B has two distinct bond types, A-B and B-B, so the output contains one distribution column for each. By default, all molecule types are used; restrict this with `-mt`. Adding `--all` generates additional per-individual-bond distributions alongside the per-type-pair ones — bond 1, bond 2, ..., in the order they appear in the structure file. This is useful when bonds of the same bead type pair are structurally inequivalent.

If the coordinate file contains molecules split across periodic boundaries, `BondLength` joins them before the calculation. Supply `--joined` if the coordinates are already whole-molecule to skip this step.

The `-n` option computes distributions of distances between arbitrary bead pairs, not just bonded ones. Pairs are given as consecutive 1-based bead indices (in the order beads appear in the structure file) after the output filename: `-n dist.txt 1 5` measures the distance between beads 1 and 5, and multiple pairs can be specified at once, e.g. `-n dist.txt 1 5 2 4`. If an index exceeds the number of beads in the molecule, the last bead is used; if no indices are given, the first and last bead of each molecule are used as the default pair. The `-w` option emits a warning whenever a bond length exceeds the given threshold.

The `-t` option writes per-timestep averages to a separate file. The columns follow the same order as in the main output file: bead type pair averages (and, if `--all` is used, individual bond averages; and, if `-n` is used, bead pair distance averages).

```
Usage:  BondLength <input> <width> <output> [options]
```

Mandatory arguments

`<input>`	input coordinate file
`<width>`	bin width for the distribution
`<output>`	output file with bond length distributions
Options	
`-mt <name(s)>`	molecule type(s) to use (default: all)
`--joined`	input already contains joined (whole-molecule) coordinates
`--all`	also write per-individual-bond distributions
`-n <file> [ints]`	write distance distributions for specified bead index pairs to `<file>`; `[ints]` are pairs of 1-based bead indices (default: first and last bead of each molecule)
`-w <float>`	warn when a bond length exceeds `<float>`
`-t <file>`	write per-timestep average bond lengths to `<file>`
`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

Format of output files:

1) `<output>` — bond length distributions grouped by bead type pair
   - 1st line: command used to generate the file
   - next lines (one per selected molecule type): comment lines listing column numbers and their bead type pair labels; if `--all` is used, the range of individual bond columns is noted at the end of each line
   - data lines: column 1 is the bin centre; remaining columns are normalised bond length distributions
   - second-to-last line: comment line mapping column numbers to bead type pairs, three columns per pair in the order min, max, average
   - last line: commented-out minimum, maximum, and average lengths for each pair
2) `-n <file>` — distance distributions for specified bead pairs
   - 1st line: command used to generate the file
   - 2nd line: bead names in order for each selected molecule type
   - next lines: comment lines listing column numbers and bead index pairs (written as smaller-index – larger-index)
   - data lines: column 1 is the bin centre; remaining columns are normalised distance distributions
   - second-to-last line: comment line mapping column numbers to bead index pairs, three columns per pair in the order min, max, average
   - last line: commented-out minimum, maximum, and average distances
3) `-t <file>` — per-timestep average bond lengths
   - 1st line: command used to generate the file
   - next lines: comment lines describing each column (bead type pairs; if `--all`, the range of individual bond columns; if `-n`, the bead pair distance columns)
   - data lines: column 1 is the timestep number; remaining columns are per-timestep mean values in the same order as in `<output>`

3.7 DensityBox

`DensityBox` calculates number density profiles along all three box axes for every bead type, producing one output file per axis.

The box is divided into slices of thickness `<width>` perpendicular to each axis in turn. The number density of bead type i in a slice at position α is

$$\rho_i(\alpha) = \frac{\langle N_i(\alpha) \rangle}{\Delta\alpha \, L_\beta \, L_\gamma}, \tag{3.12}$$

where $\langle N_i(\alpha) \rangle$ is the time-averaged number of type-i beads in the slice, $\Delta\alpha$ is the slice thickness (i.e., `<width>`), and L_β, L_γ are the box lengths along the remaining two axes.

Three output files are created automatically, named `<output>-x.txt`, `<output>-y.txt`, and `<output>-z.txt`. Bead types that never appear in the trajectory are silently omitted from the output columns.

By default, all beads — bonded and unbonded — contribute to the density. Because the utility groups them by bead type name, beads with the same name in different molecule types are indistinguishable. To obtain separate density profiles for overlapping bead types, use `-x` to exclude specific molecule types, running the utility twice with complementary exclusion sets (note that unbonded beads cannot be excluded and will always appear in both outputs).

The `--per-bead` option switches the grouping from bead type to bead position within each molecule type. Each column then represents the k-th bead of a given molecule type (column label `MolName:k`, 1-indexed), which reveals how density varies along the chain. Unbonded beads are skipped in this mode.

`DensityBox` requires an orthogonal box with constant side lengths; a triclinic box or a varying box size will produce unreliable results.

`Usage: DensityBox <input> <width> <output> [options]`

Mandatory arguments	
`<input>`	input coordinate file
`<width>`	slice thickness
`<output>`	base name for the three output files; suffixed automatically with `-x.txt`, `-y.txt`, and `-z.txt`
Options	
`-x <name(s)>`	exclude beads belonging to the specified molecule type(s)
`--per-bead`	output density per bead position within each molecule type, rather than per bead type
`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

Format of output files (`-x.txt`, `-y.txt`, `-z.txt` are identical in structure):

1) one density file per axis, e.g. `<output>-z.txt`
   - 1st line: command used to generate the file
   - 2nd line: column headers

  - column 1: bin centre (distance from the box origin along the given axis); e.g., with `<width>` = 0.5, the first centre is 0.25, the second is 0.75, etc.
  - remaining columns: number density per bead type (default) or per bead position within each molecule type (`--per-bead`), labelled by bead type name or `MolName:k` respectively
- data lines: bin centres and corresponding densities

3.8 Info

`Info` reads an input structure file, prints system composition to standard output, and can optionally write the (possibly modified) system to a structure file of any supported format.

By default, the utility prints a summary of bead types, molecule types, counts, and simulation box dimensions. Using `--verbose` additionally prints per-bead and per-molecule details. When a secondary structure or coordinate file (`-i` or `-c` option, respectively) is present as well, `--verbose` also prints the initial composition of each input before the final merged result.

Supplementing system information

A secondary structure file supplied via the `-i` option fills in bead type properties (charge, mass, radius) that are missing from the primary file, matching bead types by name. If both files share molecule types with identical names and bead type order, the secondary file can also supply bond types, angles, and related topology data absent from the primary file; note that all bead type parameters must match exactly for this to work.

The `--chbt` option goes further: instead of only filling in missing values, it remaps the bead types inside each molecule to those defined in the secondary file. Molecules are matched by name and bead count; molecule types with mismatched bead counts are skipped.

For `vtf` structure files, bead type identity is determined by bead name alone by default. The `--detailed` flag additionally uses charge, mass, and radius to distinguish bead types, so beads sharing a name but differing in other properties are assigned distinct types. For example, three `atom` lines with the same name but two different masses and one with no mass normally produce a single bead type; with `--detailed` they become three separate types (one with undefined mass).

Coordinates

Structure file formats that natively embed coordinates (`data`, `lammpstrj`, `xyz`) include them automatically; for `vtf` files, structure and coordinates may differ (not all beads in the structure block/file have to be in the coordinate block/file), so to read coordinates from a `vtf` file, the coordinate file (e.g., the same `vtf` file) must be included via the `-c` option. The `-st` option selects which timestep of the coordinate file to use (default: the first). When coordinates are available, the system is pruned to contain only the beads present in that timestep. If no coordinates are read, the

system is not pruned, and all coordinates in the output structure file (`-o` option) will be 0's.

Restructuring the system

The `--frag` option splits topologically disconnected molecules (molecules whose bonds do not form a single connected graph) into separate fragment molecule types named `<orig_name>_1`, `<orig_name>_2`, etc. Single-bead fragments are demoted to unbonded beads.

The `--mol` option converts all unbonded beads into one-bead molecules, using the bead type name as the molecule type name. This is useful when the output is to be processed by utilities that only operate on molecules (e.g., `Aggregates`).

Output structure file

When `-o` is specified, `Info` writes the final system to a structure file; the format is inferred from the file extension. Writing to `itp` or `pdb` is not supported. Only one timestep of coordinates is written.

For `vtf`/`vsf` output, `-def` specifies the bead type for the `atom default` line; the default is the bead type with the greatest number of unbonded beads.

For LAMMPS `data` output, `--mass` collapses bead types that differ only in charge into a single LAMMPS atom type in the `Mass` section while preserving per-atom charges in the `Atoms` section. The `-ebt` option appends a given number of extra dummy atom types (mass 1, absent from the `Atoms` section), useful for, e.g., SRP ghost particles that require pre-declared atom type slots.

Library-based bead renaming

The `-lib` option points to a library directory from which bead type names are reassigned and DPD interaction parameters are looked up. The resulting interaction table is printed to standard output in addition to the system composition. When the output is a `FIELD` file (`-o` option), the interaction block is also appended to that file.

Bead types are renamed in two passes. Molecule types are matched to library molecules by name and bead count, and each of their beads takes the name, charge, mass, and radius of the corresponding library bead. Unbonded beads have no name to match on in, e.g., a LAMMPS `data` file, so they are matched by charge, and by count where the count is predictable—a counterion of a molecule present in the system is expected exactly (number of parent molecules) $\times$ (counterion count) times. A charge-and-count match wins over a charge alone, and an ambiguous match renames nothing, on the grounds that a wrong name is worse than no name.

If molecule types in the input file are unnamed, `-sys` supplies a `system_info` file to assign names to them before library renaming. Without `-sys`, unnamed molecule types cause the renaming step to be skipped with a warning.

Asking about the library itself

Two options ask about the library rather than about a system. Both require `-lib`, take no input file, and exit once they have printed their answer.

`--mol-info <mol>` prints what one library molecule is, as `key value` lines in the library's own grammar: its `name` and `role`, its `M_w` (only when the file gives one), the number of beads, the total charge, the number of ions (the sum of q^2 over its beads), the smallest and largest z-coordinate of its template, and one `counterion` line per declared counterion with the counterion's name, count, bead count, charge, and ion count. It is meant to be parsed by build scripts, so they do not have to re-implement the library format to work out how many beads a molecule brings with it.

`--check-library` checks the whole library for the things merely loading it does not: molecule files whose `name` disagrees with the filename, names declared twice, counterions with no file, bead/bond/angle IDs with no entry in the index files, molecules that do not come out neutral with their counterions, and row indices that have drifted out of step with their position in the block. It prints one line per problem plus a summary, and exits with 1 if anything was found, so it can be used in a test suite. A species that is somebody else's counterion—or that has no `M_w`—is exempt from the charge check, since a bare ion is charged by definition and is neutralised by whatever names it. A file that cannot be parsed at all is still fatal, as it is anywhere else; this checks what a library that loads can still get wrong.

TODO: Describe the `system_info` file format, or cross-referenc the relevant section once written.

TODO: Add `-lib`/`-sys` examples to `Examples/Info`.

Usage: `Info <input> [options]`

Mandatory argument	
`<input>`	input structure
Options for input files	
`-i <file> [type]`	secondary structure file supplying additional information
`-c <file>`	input coordinate file
`--detailed`	(`vtf` input only) identify bead types by name, charge, mass, and radius rather than name alone
Options for system modification	
`--chbt`	replace bead types in molecules using the `-i` file; molecules matched by name and bead count
`--frag`	split disconnected molecules into fragment molecule types; single-bead fragments become unbonded beads
`--mol`	convert unbonded beads into one-bead molecules
`--unique`	make all bead and molecule names unique
Options for output file	
`-o <file>`	output structure file (`itp` and `pdb` output not supported)

`-def <bead>`	(`vtf`/`vsf` output only) bead type for the `atom default` line (default: type with the most unbonded beads)
`--mass`	(`data` output only) define LAMMPS atom types by mass and print per-atom charges in the `Atoms` section
`-ebt <int>`	(`data` output only) number of extra dummy atom types to add
Library options	
`-lib <dir>`	library directory for bead type renaming and DPD interaction lookup
`-sys <file>`	`system_info` file to name unnamed molecule types before library renaming
`--mol-info <mol>`	print one library molecule's role, mass, bead count, charge, z-extent, and counterions, then exit (requires `-lib`; takes no input file)
`--check-library`	check the library for bad names, dangling counterions, unknown type IDs, unbalanced charges, and drifted row numbering, then exit (requires `-lib`; takes no input file)
Other options (see the beginning of Chapter 3)	
`-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

3.9 JoinSystems

`JoinSystems` merges two coordinate files into a single new system containing all beads from both.

With no options the two systems are placed as-is: the output simulation box is the smallest orthogonal box that simultaneously encompasses both input boxes, and all bead coordinates are left unchanged (figure 3.3b). The input files can have different lower-bound offsets; `JoinSystems` reads those bounds and aligns the two systems correctly before computing the output box.

Offsetting the second system (`-off`). The second system can be shifted relative to the first using `-off <x> <y> <z>`, where each component is either a number or `c`. A numeric value displaces all beads in the second system by that amount in the given direction. Without `--real`, the value is a fraction of the *first* system's box length in that direction; with `--real` it is in absolute coordinate units. Specifying `c` for a direction aligns the centre of the second system's box with the centre of the first system's box in that direction (figure 3.3d), regardless of `--real`. The output box is the smallest encompassing box computed *after* the offset is applied.

Overriding the output box (`-b`). `-b <x> <y> <z>` replaces the automatically computed encompassing box with an explicitly specified one. The new box is centred on the centre of the encompassing box (figure 3.3c), so bead coordinates do not move. No check is performed to verify that all beads fall within the new box. `-off` and `-b`

can be combined: the offset is applied first, the encompassing box is computed, and then `-b` overrides it (figure 3.3e).

Output files. The output can be any supported coordinate format. An additional structure file can be produced with `-o`. Note that some formats (`lammpstrj`) store explicit box bounds, while others (`vtf`) store only sidelengths; when the output is `vtf`, all coordinates are shifted so the box origin is at zero, which changes the absolute positions of beads relative to the input.

Per-file input options. Because two input files are used, the usual structure-file (`-i`) and starting-timestep (`-st`) options are suffixed with `1` or `2` to indicate which file they apply to. For LAMMPS `data` input, the starting-step option has no effect since those files contain only a single timestep.

These examples are available in the `Examples/JoinSystems` directory.

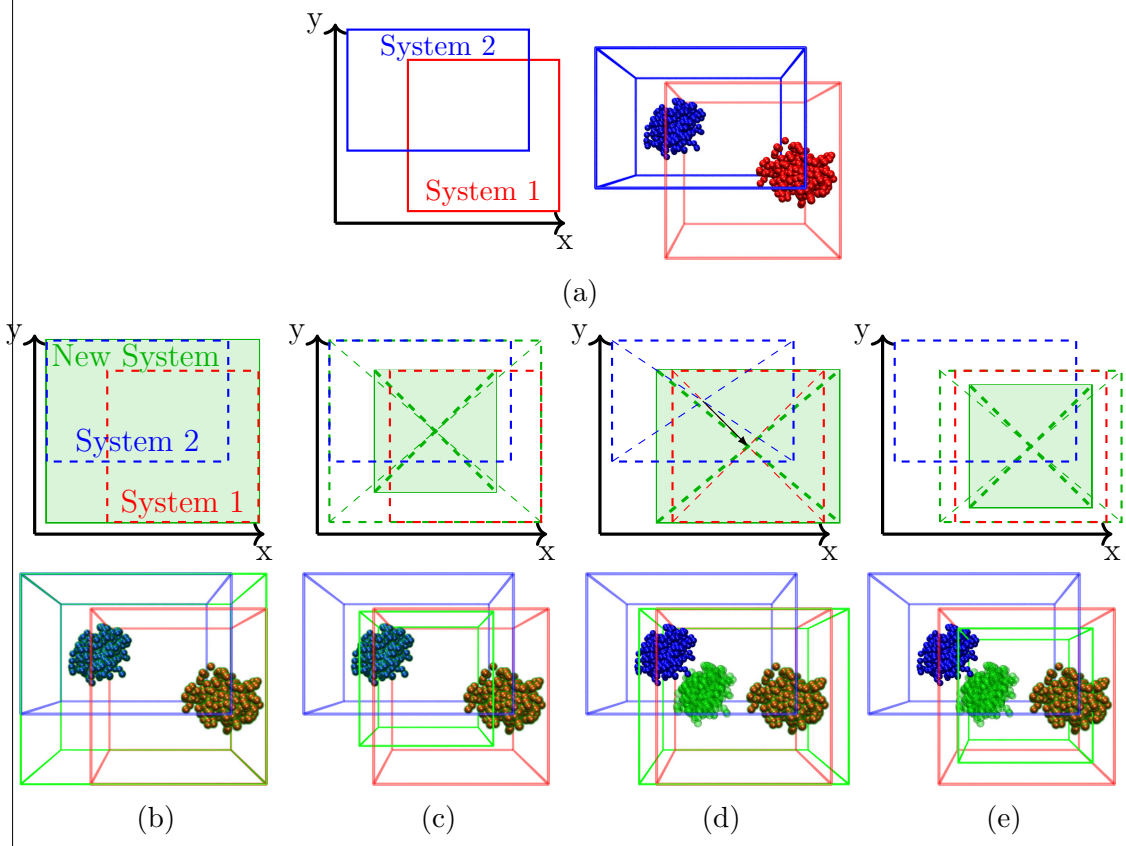


Figure 3.3: Schematic representation and corresponding snapshots illustrating `JoinSystems` behaviour. (a) The original systems: `System1.lammpstrj` (red) and `System2.lammpstrj` (blue) in the `Examples/JoinSystems` folder. The rest show examples of possible options (green rectangles and transparent balls represent the new systems): (b) no options; (c) `-b 20 20 20` option; (d) `-off c c c`; (e) `-b 20 20 20 -off c c c`, i.e., combining (c) and (d). These new systems are in the `Examples/JoinSystems` folder as `NewSystem\_\#.lammpstrj`, where `\#` is `b`, `c`, `d`, or `e`.

Usage: `JoinSystems <input1> <input2> <output> [options]`	
Mandatory arguments	
`<input1>`	first input coordinate file
`<input2>`	second input coordinate file
`<output>`	output coordinate file for the merged system
Options	
`-off <x>|c <y>|c <z>|c`	shift the second system; each component is a fraction of the first system's box length (or absolute with `--real`), or `c` to align box centres
`-b <x> <y> <z>`	override the output simulation box sidelengths; box centred on the smallest encompassing box
`--real`	treat `-off` values as absolute coordinates (default: fraction of first system's box length)
`-o <file>`	write an additional output structure file
As two input files are read, some options are suffixed with `1` or `2` to indicate which file they apply to:	
`-i1`/`-i2`, `-st1`/`-st2`, `-ft`, `--verbose`, `--silent`, `--help`, `--version`	

TODO: The `--real` flag is listed as applying to both `-b` and `-off` in the help text, but `-b` is currently always in absolute units. Verify whether fractional `-b` values (scaled to the first system's box) are intended to be supported.

3.10 PairCorrel

`PairCorrel` calculates the radial distribution function (RDF, also called the pair correlation function) $g(r)$ between all pairs of specified bead types and writes them to a single output file.

The RDF is defined such that $g_{ij}(r)\,4\pi r^2\,\mathrm{d}r$ gives the mean number of beads of type j in a shell $[r,\, r+\mathrm{d}r)$ around a bead of type i, relative to what an ideal gas at the same bulk density would give. For a simulation box of volume V averaged over T timesteps:

$$g_{ij}(r) = \frac{V \cdot c_{ij}(r)}{4\pi r^2\,\Delta r \cdot N_{ij} \cdot T}, \tag{3.13}$$

where $c_{ij}(r)$ is the total pair count in the bin centred at r, Δr is the bin width, and N_{ij} is the number of distinct ordered or unordered pairs: $N_{ij} = N_i N_j$ for unlike types ($i \neq j$) and $N_{ij} = N_i(N_i - 1)/2$ for like types ($i = j$).

By default all bead types are used and every pairwise combination is computed. Use `-bt` to restrict to specific types; specifying types A and B produces three columns: $g_{\text{A-A}}$, $g_{\text{A-B}}$, and $g_{\text{B-B}}$. Beads of the same type are pooled regardless of molecule. Columns appear in type-index order (the order of bead types in the structure file), not the order given on the command line.

When only a small number of specific cross-pair RDFs are needed, `--pairs` avoids computing the full upper-triangular set: `-bt` then takes consecutive name pairs, so `-bt A B C D` selects only A–B and C–D. `-bt` is mandatory with `--pairs`.

The default maximum distance is one third of the shortest box side length. The normalisation corrects for shells truncated by the box boundary, so larger values via `-d` are valid, though accuracy degrades as fewer complete shells contribute.

Usage: `PairCorrel <input> <width> <output> [options]`

Mandatory arguments	
`<input>`	input coordinate file
`<width>`	bin width Δr
`<output>`	output file

Options	
`-bt <name(s)>`	bead types to include (default: all); with `--pairs`, names are read as consecutive pairs
`--pairs`	`-bt` specifies explicit pairs A B [C D ...] rather than individual types; requires `-bt`
`-d <float>`	maximum distance for RDF calculation (default: $\frac{1}{3}\min(L_x, L_y, L_z)$; in-plane box sides only with `-D2`)
`-D2 <axis>`	assume a 2D system (e.g., slit) that is non-periodic in the `x`, `y`, or `z` direction: distances are measured in-plane only and the RDF is normalised by area instead of volume
`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`	

Format of `<output>`:

- 1st line: command used to generate the file
- 2nd line: column headers — (1) bin centre r; one column per selected pair in upper-triangular type-index order, labelled `<type\_i>-<type\_j>`
- subsequent lines: r and $g_{ij}(r)$ values, one bin per line; bin centre is at $r = (k+\frac{1}{2})\Delta r$ for bin k

TODO: The `-d` option accepts any positive value, but the shell-volum correction assumes a cubic box (it subtracts 6 spherical caps). For non-cubi boxes or large `-d`, the correction becomes approximate.

3.11 PersistenceLength

`PersistenceLength` computes orientational correlation functions of the bond vectors along linear chains, from which the persistence length l_p can be extracted (the utility itself does not fit — it outputs the correlation data you then fit). The chain must have ordered bead ids, i.e. bead order 1-2-3-... giving bonds 1-2, 2-3, ...

For each bond i let $\hat{\mathbf{u}}_i$ be its unit vector. As a function of the bond separation s (an integer number of bonds), three observables are calculated, each averaged over

all selected molecules and timesteps:

$$S_2(s) = \left\langle \hat{\mathbf{u}}_i \cdot \hat{\mathbf{u}}_{i+s} \right\rangle_i = \left\langle \cos\theta(s) \right\rangle \xrightarrow{\text{WLC}} \exp\!\left(-\tfrac{s\,\langle l_b\rangle}{l_p}\right), \tag{3.14}$$

$$S_1(s) = \left\langle \hat{\mathbf{u}}_b \cdot \hat{\mathbf{u}}_{b+s} \right\rangle, \tag{3.15}$$

$$S_3(s) = \left\langle\, |\mathbf{r}_a - \mathbf{r}_c|^2 \,\right\rangle = \left\langle R^2(s{+}1) \right\rangle, \tag{3.16}$$

where $\langle l_b \rangle$ is the mean bond length. S_2 is the standard bond-orientational autocorrelation, averaged over every starting bond i; its running sum $\langle l_b \rangle \sum_s S_2(s)$ estimates l_p. S_1 is the same cosine but measured against a *fixed* reference bond b — the first bond ("`S1`") and, separately, the last bond ("`S1 (rev)`"); this is Flory's definition, with $l_p = \langle l_b \rangle \sum_s S_1(s)$ (the "`sum S1`" columns). S_3 is the mean-square distance R^2 across a subchain of $s+1$ consecutive bonds (from the first bead of bond i to the last bead of bond $i+s$), fit to the worm-like-chain $\langle R^2 \rangle$.

By default all molecule types are used; restrict this with `-m`. Each type is treated separately (results are per type), averaging over its molecules and over all timesteps. $S_1(0)$ and $S_2(0)$ are 1 by construction.

The three estimators are redundant on purpose — they trade off differently against finite-chain end effects. S_2 (lag-averaged) is the most robust; S_1 is more sensitive to the chain ends, which is why it is reported from both ends so you can gauge that bias. All three should give a consistent l_p for a long enough, well-equilibrated chain.

Use `-ns`/`-ne` to analyse only part of each molecule: they set the first and last bead (1-based, in structure-file order) of the sub-chain to correlate, and at least three beads must remain. This is useful to drop chain ends. Note an asymmetry: the forward S_1/S_2 honour `-ne`, but the reverse S_1 is always measured from the true last bond of the molecule, so with `-ne` set the forward and reverse curves cover different bond ranges.

If the coordinate file has molecules split across periodic boundaries, `PersistenceLength` joins them first; pass `--joined` if the coordinates are already whole-molecule.

TODO: Add an Examples/ entry: persistence length of a semiflexible bead-sprin chain, showing the exponential fit of `S2` and the `sum S1`/`sum S2` plateau giving l_p.

`Usage: PersistenceLength <input> <output> [options]`

Mandatory arguments	
`<input>`	input coordinate file (structure file must define bonds)
`<output>`	output file with the correlation functions

Options	
`-m <name(s)>`	molecule type(s) to use (default: all)
`--joined`	input already contains joined (whole-molecule) coordinates
`-ns <int>`	start with the `<int>`-th bead of each molecule (default: 1)
`-ne <int>`	end with the `<int>`-th bead of each molecule (default: last); at least three beads must remain

`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`

Format of output files:

1) `<output>` — correlation functions versus bond separation
- 1st line: command used to generate the file
- 2nd line: comment line listing, per molecule type, the seven data columns: `S1`, `sum S1`, `S1 (rev)`, `sum S1 (rev)`, `S2 (autocorr)`, `sum S2 (autocorr)`, `S3`
- 3rd line: comment line mapping column numbers to molecule type names (column 1 is the lag; then seven columns per selected type)
- data lines: column 1 is the lag s (bond separation, in bonds — multiply by the mean bond length for a contour length); the remaining columns are the seven observables above for each molecule type
- molecule types shorter than the longest one contain `nan` in the rows beyond their length (no bond pairs at that separation)

TODO: Source: the utility is marked "very messy" and output handling is still provisional (`// TODO: output handling`); the mean bond length is accumulated internally but not written to the output.

3.12 Selected

This utility creates a new coordinate file. By default, the utility saves all bead and molecules types (basically transforming one coordinate file format into another), but there are options to fine-tune what is excluded or kept (`-bt`, `-mt`, `--keep`, `-cx/y/z`). There are also some basic manipulations for the coordinates from connecting molecules split via periodic boundary conditions or wrapping beads back into the box (`--join`, `--keep`) to scaling coordinates or moving particles (`-sc`, `-m`).

Selecting bead and/or molecule types to exclude or keep

The `-bt` and `-mt` options select bead and molecule types, respectively. By default, the selected types are excluded (i.e., all other types are saved into the output file); the `--keep` option reverses the behaviour (i.e., only the selected types are saved).

Table 3.12: Interactions of `-bt`, `-mt`, and `--keep` options when used on a system containing bead types that are both unbonded and in molecules. For molecule types, the name represents initial composition (e.g., `ABC` comprises 1 bead of each of `A`, `B`, and `C` bead types), and the brackets show the final composition of the molecule (e.g., 1 (no `B`) for `ABC` molecule means only `A` and `C` beads remain in the molecule). For bead types, $a + b$ indicates unbonded beads (a) and beads in molecules (b).

Options used on 'full'	Molecule types			Bead types		
	ABC	AB	AC	A	B	C
full	1	1	1	10+3	10+2	10+2
`-bt B`	1 (no B)	1 (no B)	1	10+3	0	10+2
`-bt B -mt AB`	1 (no B)	0	1	10+2	0	10+2
`-bt B -mt AB --keep`	1 (only B)	1	0	0+2	10+2	0

Table 3.12 shows possible interactions between the three options when some

beads are unbonded and other beads of the same type are in a bond. The 'full' system is in the `Examples/Selected` directory as the `full.vtf` file.

Constraining and manipulating coordinates

The `-cx`, `-cy`, and `-cz` options constrain beads to be saved to specified coordinate ranges (by default, the range is in fractions of the simulation box). The options accept multiples of number pairs for each axis; a bead coordinate must fall within at least one of the ranges, e.g., using `-cx 0 0.1 0.9 1` would save beads whose x coordinates are within 0 to 10% or 90% to 100% of the box size in the x-direction. Adding, e.g., `-cy 0.4 0.6` would save only the beads that fulfil the `-cx` option as well as this one, i.e., that y coordinate is between 40% and 60% of the simulation box.

The `-sc <float>` option scales all coordinates by dividing them via the specified number.

The `-m <x> <y> <z>` option moves all coordinates by the specified vector (by default, the vector is in fractions of simulation box).

If multiple options are used, they manipulate the coordinates in the order `-cx/y/z`, `-sc`, `-m`, i.e. first, coordinates are constrained, then scaled, and lastly moved. See figure 3.4 for an example: if all three options are specified in one command, it transforms the 'original' system in the direction of 'all options at once' with the beads ending up outside the box. This is equivalent to running three consecutive commands, first only with `-cx/y/z` (from 'original' system to the right), then with `-sc` (down), and lastly with `-m` (left). Figure 3.4 shows what happens when the `Selected` commands are run in different order. These examples are also in the `Examples/Selected` directory.

As mentioned above, both `-cx/y/z` and `-m` options accept fractions of box size (i.e., numbers between 0 and 1) by default. To specify 'real' coordinates instead, use the `--real` option.

Note that neither of these options changes the size of the simulation box.

Dealing with periodic boundary conditions

The input system may contain beads that are outside the simulation box. The `--wrap` option returns all beads into the simulation box...

Conversely, molecules can be disconnected due to the periodic boundary conditions with one part of the molecule on one side of the box and another part on the opposite side. The `--join` removes the periodic boundary conditions, making the molecule whole as well as stick out of the simulation box. The centre of mass of every 'joined' molecule is inside the simulation box.

If both options are specified, the coordinates are first wrapped via `--wrap` and then the molecules are 'joined' via `--join`.

Usage: `Selected <input> <output> <bead type(s)> [options]`

Mandatory arguments	
`<input>`	input coordinate file

<output>	output coordinate file
Options	
-bt <bead type>	bead types to exclude
-mt <mol type>	molecule types to exclude
--keep	save only specified types instead of excluding them
--join	join molecules by removing periodic boundary conditions
--wrap	wrap simulation box (i.e., apply periodic boundary conditions)
-n <int(s)>	save only specified timesteps
--last	save only the last step
-sc <float>	divide all coordinates by given value
-m <x> <y> <z>	add specified vector to all coordinates (-sc is applied first)
-cx n*2*<float>	constrain x-coordinates to specified dimension; multiple pairs possible
-cy n*2*<float>	same as -cx but with y-coordinates
-cz n*2*<float>	same as -cx but with z-coordinates
--real	use real coordinates instead of fractions of box size (for -cx/y/z and -m options)
Other options (see the beginning of Chapter 3)	
-st, -e, -sk, -i, --verbose, --silent, --help, --version	

3.13 Surface

Surface identifies the two surfaces of a layered structure (e.g. a polymer brush or a lipid bilayer) and writes their time-averaged coordinates and areas.

The algorithm is based on the Identification of the Truly Interfacial Molecules (ITIM) method TODO: add citation: Pártay et al., J. Comput. Chem. 29:945 (2008). A grid of probe spheres of radius r (set via -r, default 0.5) is placed in the plane perpendicular to <axis>, with a spacing of <width> between adjacent probes. For each probe, the bead whose sphere (of radius R_k, defaulting to 0.5 if not specified in the structure file) overlaps the probe sphere and whose coordinate along <axis> is most extreme defines the surface at that grid point. Two surfaces are found per probe position (labelled 'surface 1' and 'surface 2'), corresponding to the two sides of the structure.

The utility operates in two modes:

- **Default (from the box centre):** probes approach from the box centre outward towards both walls. This finds the innermost extent of each layer, i.e. the surface facing the solvent gap between the layers. Use this for structures such as a polymer brush grafted on both walls.
- **--in (from the box edges):** probes approach from both walls inward towards the centre. This finds the outermost extent of a structure sitting inside the box, i.e. the outer leaflets. Use this for structures such as a lipid bilayer.

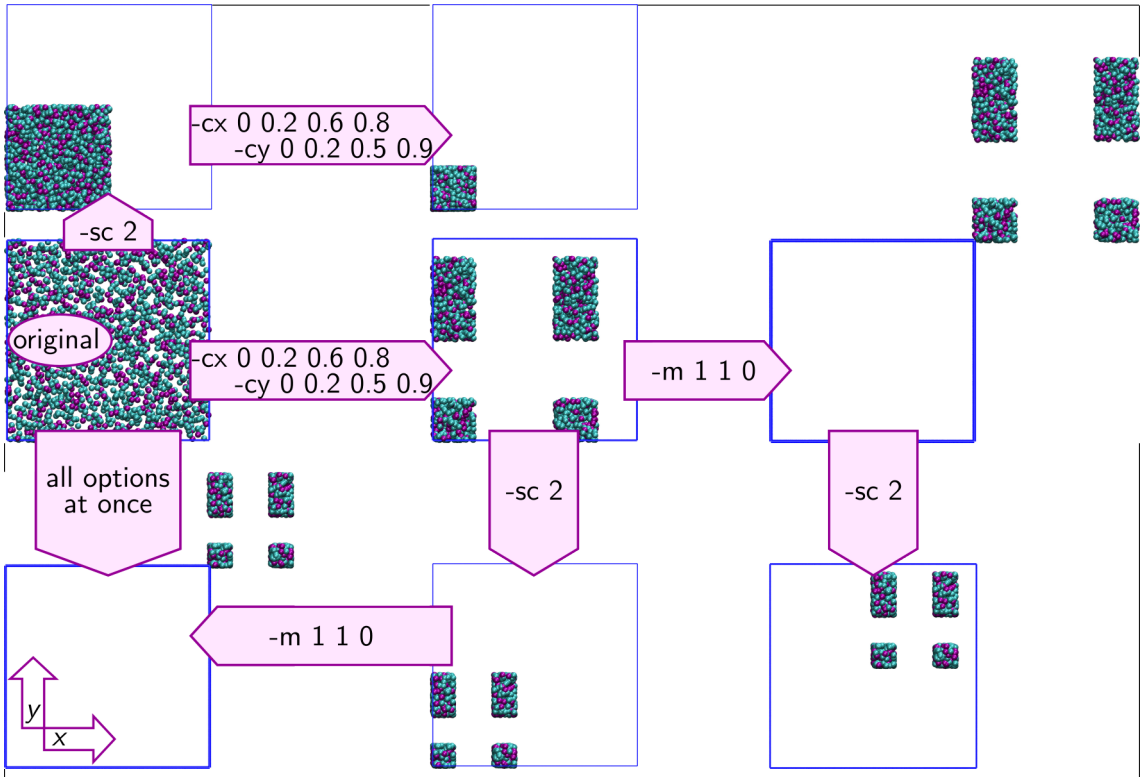
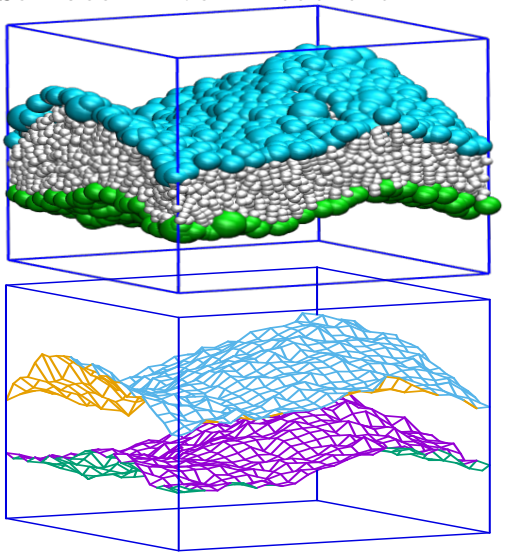


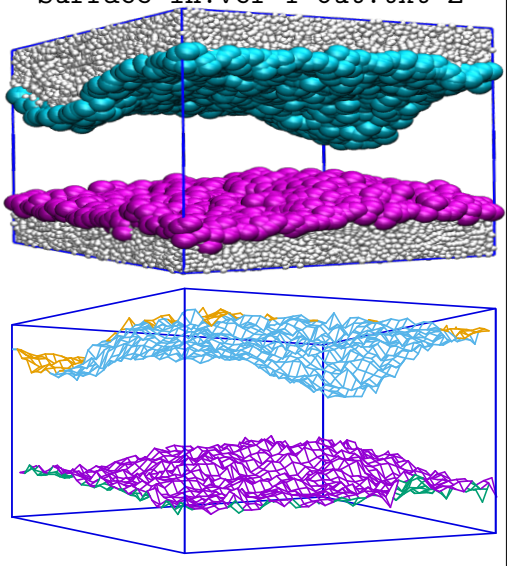
Figure 3.4: Some of the possible results when combining sequential runs of the `Selected` utility, namely (1) constraining x and y coordinates via `-cx 0 0.2 0.6 0.8 -cy 0 0.2 0.5 0.9`, (2) scaling (dividing) coordinates via `-sc 2`, and (3) moving coordinates via `-m 1 1 0`. The blue lines represent the simulation box (the varying line thickness is just an artefact of how the snapshots were created in `vmd`).

The two examples below illustrate each mode (top: snapshot with surface beads highlighted; bottom: surface coordinates output):

`Surface in.vcf 1 out.txt z --in`



`Surface in.vcf 1 out.txt z`



By default all beads contribute to the surface search. Use `--bonded` to restrict this to beads in molecules, or `-bt` to restrict to specific bead types. If both `--bonded`

and `-bt` are given, `--bonded` takes precedence. Beads without a radius defined in the structure file receive radius 0.5 automatically, and a warning is printed.

The surface area is computed by triangulating the grid: each pair of adjacent grid cells is split into two triangles using Heron's formula, and the triangle areas are summed and scaled to the actual box area. Three areas are reported per timestep: surface 1, surface 2, and a 'middle surface' at their midpoint. Use `-a` to write these per-timestep areas; the overall averages are appended at the end of that file.

The `-wd` option writes the distribution of surface thickness (i.e. the distance $|\text{surface}\,1 - \text{surface}\,2|$ at each grid point and timestep) to a file. The `-w` option writes the per-timestep average thickness. TODO: Verify whether - can be used independently of -wd source TODO flags a possible inconsistency between the two.

The `-b` option saves the detected surface beads to a coordinate file at each timestep. Beads that are present in the trajectory but not identified as surface beads are written with coordinates $(0, 0, 0)$ so that they remain invisible in typical visualisation tools without disturbing the frame count.

`Usage: Surface <input> <width> <surf.txt> <axis> [options]`

Mandatory arguments	
`<input>`	input coordinate file
`<width>`	probe grid spacing (side length of each grid square)
`<surf.txt>`	output file with time-averaged surface coordinates
`<axis>`	axis normal to the surface: `x`, `y`, or `z`

Options	
`--in`	search from box edges inward (for bilayers; default searches from box centre outward, for brushes)
`--bonded`	use only beads belonging to molecules
`-bt <name(s)>`	use only the specified bead type(s)
`-r <float>`	probe radius (default: 0.5)
`-a <area.txt>`	write per-timestep surface areas to `<area.txt>`
`-wd <file> <w>`	write distribution of surface thickness to `<file>`; `<w>` is the bin width
`-w <file>`	write per-timestep average surface thickness to `<file>`
`-b <file>`	save surface beads to a coordinate file at each timestep

`-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version`

Format of output files:

1) `<surf.txt>` — time-averaged surface coordinates on the 2D grid
   - 1st line: command used to generate the file
   - 2nd line: column headers: the two in-plane coordinates (columns 1–2), surface 1, surface 2, and average surface coordinates along `<axis>` (columns 3–5)
   - data block: one line per grid point; the grid varies the second in-plane coordinate (inner loop) and the first (outer loop), with a blank line separating each row of the outer coordinate
2) `-a <area.txt>` — per-timestep surface areas
   - 1st line: command used to generate the file

- 2nd line: column headers: (1) timestep, (2) area of surface 1, (3) area of surface 2, (4) area of the middle surface
- data lines: one per timestep
- last two lines: overall average areas (header then values) for surface 1, surface 2, and the middle surface

3) `-wd <file>` — distribution of surface thickness
- 1st line: command used to generate the file
- 2nd line: column headers: (1) bin centre, (2) normalised count
- data lines: thickness distribution
- last line: commented-out overall average thickness

4) `-w <file>` — per-timestep average surface thickness
- 1st line: command used to generate the file
- 2nd line: column headers: (1) timestep, (2) thickness
- data lines: one per timestep
- if `-wd` is also used: overall average thickness appended at the end as a comment

5) `-b <file>` — coordinate file with surface beads per timestep; non-surface beads present in the timestep are written with coordinates $(0, 0, 0)$